অধ্যায় ১৭

১৭

JavaScript Regular Expressions, Advanced Validation এবং Secure Input Processing

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Character Class, Quantifier (Greedy ও Lazy) এবং Anchor দিয়ে নির্ভুল Pattern লিখতে।
- Capturing, Named ও Non-capturing Group, Backreference এবং Alternation ব্যবহার করতে।
- Lookahead ও Lookbehind দিয়ে "পাশে কী আছে" শর্ত প্রকাশ করতে।
- `exec`, `matchAll`, Function-সহ `replace`, Regex দিয়ে `split` এবং Sticky Flag ব্যবহার করতে, এবং `lastIndex`-এর ফাঁদ এড়াতে।
- ব্যবহারকারীর লেখা দিয়ে নিরাপদে Regex তৈরি করতে (Escaping)।
- বাংলা লেখা ও বাংলা সংখ্যা Unicode-সচেতনভাবে যাচাই ও রূপান্তর করতে।
- "আগে Normalize, তারপর Validate, তারপর Parse"—এই ক্রমে Input প্রক্রিয়া করতে।
- Email, Phone ও Password যাচাইয়ের সীমা বুঝতে, এবং Regex-এর বাড়াবাড়ি এড়াতে।
- ReDoS (Regular Expression Denial of Service) চিনতে ও প্রতিরোধ করতে।
- একটি সম্পূর্ণ **FormShield** Schema-ভিত্তিক Validation Engine তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৮: Regex-এর পরিচয় (`test`, `match`, `replace`, `\d`, `\s`, `+`, `{n}`, `^`, `$`, Flag `g`, `i`, `u`, `\p{L}`), Unicode Normalization এবং XSS।
- অধ্যায় ১১: Trust Boundary, `ValidationError`, সব ভুল একসঙ্গে জানানো।
- অধ্যায় ১২: ES Module।
- অধ্যায় ১৫: `URL` দিয়ে ঠিকানা যাচাই।

এই অধ্যায় কেন গুরুত্বপূর্ণ

অধ্যায় ৮-এ Regular Expression-এর সঙ্গে পরিচয় হয়েছে, এবং তারপর থেকে প্রায় প্রতিটি Project-এ ছোট ছোট Pattern দেখেছেন: Email-এর গঠন, পরিমাণের পূর্ণসংখ্যা, মোবাইল নম্বর। কিন্তু বাস্তব Input-এর চেহারা অনেক বেশি এলোমেলো: কেউ নম্বর লেখেন `+880 1712-345678`, কেউ `০১৭১২৩৪৫৬৭৮`; কেউ তারিখ লেখেন `2026-02-30`, যা দেখতে ঠিক কিন্তু বাস্তবে নেই। এগুলো সামলাতে Regex-এর গভীরতর ক্ষমতা লাগে।

একই সঙ্গে Regex একটি নিরাপত্তা-ঝুঁকিও: ভুলভাবে লেখা একটি Pattern একটি সাধারণ দেখতে লম্বা Input-এ Server-কে কয়েক মিনিট আটকে রাখতে পারে। আর "Regex দিয়ে বিপজ্জনক HTML পরিষ্কার" করার চেষ্টা প্রায় সব সময়ই ব্যর্থ হয়। এই অধ্যায় শেখায় Regex কখন সঠিক হাতিয়ার, কখন নয়, এবং Input প্রক্রিয়ার একটি নিরাপদ, পরীক্ষাযোগ্য পদ্ধতি।

১৭.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা (অধ্যায় ৮)	এই অধ্যায়ে নতুন
`\d`, `\s`, `+`, `{n}`, `^`, `$`, `[abc]`	`\w`, `\b`, Negated Class, `{m,n}`, Lazy Quantifier
`test`, `match`, `replace`	`exec`, `matchAll`, Function-সহ `replace`, `split`, Sticky
Flag `g`, `i`, `u`	Flag `m`, `s`, `y`, `d`; `lastIndex`-এর আচরণ
`\p{L}`, `\p{M}`, `\p{N}`	`\p{Script=Bengali}`, বাংলা সংখ্যা রূপান্তর
—	Group, Named Group, Backreference, Lookaround
—	Escaping, ReDoS, Schema-ভিত্তিক যাচাই

অধ্যায় ৮-এর জিনিসগুলো এখানে আবার শেখানো হবে না; প্রয়োজনে সেখানে ফিরে দেখুন।

১৭.২ Character Class, Quantifier এবং সীমানা

classes.js · JavaScript — চলবে: Node.js

```
const text = 'Order A-102, B-7 এবং C-3051; মোট 3টি';

console.log(text.match(/[A-Z]-\d{3,4}/g));
console.log(text.match(/[^\s,;]+/g).length);
console.log('cat category scat'.match(/\bcat\b/g));
console.log('price_2026 মূল্য'.match(/\w+/g));
```

Output

```
[ 'A-102', 'C-3051' ]
7
[ 'cat' ]
[ 'price_2026' ]
```

- `{3,4}` আগের জিনিসটি ৩ থেকে ৪ বার; `{2,}` মানে কমপক্ষে ২ বার। `B-7` বাদ পড়েছে, কারণ মাত্র একটি সংখ্যা।
- `[^...]` (Negated Class) বন্ধনীর ভেতরের অক্ষরগুলো **ছাড়া** যেকোনো একটি।
- `\b` শব্দের সীমানা: একটি শব্দ-অক্ষর (`\w`) এবং একটি অ-শব্দ-অক্ষরের মাঝখানের অদৃশ্য জায়গা। `category` বা `scat`-এর ভেতরের `cat` মেলেনি।
- `\w` মানে `[A-Za-z0-9_]`; বাংলা অক্ষর এতে নেই, তাই `মূল্য` আসেনি। `\b`-ও একই কারণে বাংলায় নির্ভরযোগ্য নয় (অনুচ্ছেদ ১৭.৬)।

Greedy এবং Lazy

greedy.js · JavaScript — চলবে: Browser Console ও Node.js

```
const html = '<b>সতর্ক</b> এবং <i>দ্রুত</i>';

console.log(html.match(/<.+>/)[0]);
console.log(html.match(/<.+?>/)[0]);
console.log(html.match(/<[^>]+>/g).join(' '));
```

Output

```
<b>সতর্ক</b> এবং <i>দ্রুত</i>
<b>
<b> </b> <i> </i>
```

Quantifier (`+`, `*`, `{m,n}`) স্বাভাবিকভাবে **Greedy**: যতটা সম্ভব বেশি নেয়, তারপর মেলাতে প্রয়োজনে পিছিয়ে আসে। তাই `<.+>` প্রথম `<` থেকে **শেষ** `>` পর্যন্ত নিয়েছে। পেছনে `?` দিলে **Lazy**: যতটা সম্ভব কম। তৃতীয় লাইনের Negated Class (`[^>]+`) প্রায়ই Lazy-এর চেয়ে ভালো: অর্থ স্পষ্ট ("`>` ছাড়া যেকোনো কিছু"), এবং পিছিয়ে আসার কাজ কম।

> **সতর্কতা:** এই উদাহরণ HTML **খোঁজার** কৌশল দেখায়, HTML **পরিষ্কার** করার নয়। Regex দিয়ে বিপজ্জনক HTML ছেঁকে ফেলা নিরাপদ নয় (অনুচ্ছেদ ১৭.১০)।

১৭.৩ Group, Backreference এবং Alternation

বন্ধনী `( )` তিনটি কাজ করে: কয়েকটি অংশকে একসঙ্গে Quantifier দেয়, মিলে যাওয়া অংশ আলাদা করে ধরে রাখে (**Capture**), এবং Alternation-এর সীমা ঠিক করে।

groups.js · JavaScript চলবে: Node.js

```
const date = /^(\d{4})-(\d{2})-(\d{2})$/.exec('2026-10-15');
console.log(date[0], date[1], date[3]);

const named = /^(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})$/.exec('2026-10-15');
const { year, month, day } = named.groups;
console.log(`${day}/${month}/${year}`);

console.log('ha-ha-ha!'.match(/(?:ha-?)+/)[0]);
console.log(/^(\w)\w*\1$/.test('level'), /^(\w)\w*\1$/.test('lever'));

console.log(/^cat|dog$/.test('catalog'), /^(?:cat|dog)$/.test('catalog'));
```

Output

```
2026-10-15 2026 15
15/10/2026
ha-ha-ha
true false
true false
```

- **Capturing Group** ফলাফলে নম্বর অনুযায়ী আসে: `[1]`, `[2]`… `[0]` পুরো মিল।
- **Named Group** `(?<name>...)` ফলাফলের `groups` Object-এ নাম ধরে আসে; Destructuring (অধ্যায় ৭) দিয়ে পড়া সহজ, এবং Pattern বদলালে নম্বর গুলিয়ে যাওয়ার ভয় নেই।
- **Non-capturing Group** `(?:...)` শুধু দলবদ্ধ করে, কিছু ধরে রাখে না।
- **Backreference** `\1` মানে "প্রথম Group যা মিলেছিল, ঠিক সেটিই আবার"। Named Group-এর জন্য `\k<name>`।
- **Alternation `|` -এর অগ্রাধিকার সবচেয়ে কম।** `^cat|dog$` মানে "`^cat`" অথবা "`dog$`", তাই `catalog` মিলে গেছে! পুরো Alternation-কে Anchor-এর ভেতরে রাখতে Group লাগে: `^(?:cat|dog)$`। এটি Validation-এর সবচেয়ে প্রচলিত ও বিপজ্জনক ভুলগুলোর একটি।

১৭.৪ Lookahead এবং Lookbehind

কখনো শর্ত থাকে "পরে অমুক আছে" বা "আগে অমুক নেই", কিন্তু সেই অমুক মিলের অংশ নয়। এগুলো **Lookaround**: অবস্থান যাচাই করে, কোনো অক্ষর গ্রাস করে না।

লেখা	নাম	অর্থ
`X(?=Y)`	Positive Lookahead	X, যার পরে Y আছে
`X(?!Y)`	Negative Lookahead	X, যার পরে Y নেই
`(?<=Y)X`	Positive Lookbehind	X, যার আগে Y আছে

লেখা	নাম	অর্থ
`(?<!Y)X`	Negative Lookbehind	X, যার আগে Y নেই

lookaround.js · JavaScript চলবে: Browser Console ও Node.js

```
const prices = 'Pen ৳50, Book ৳450, Bag $30';
console.log(prices.match(/(?<=৳)\d+/g).join(', '));

const withCommas = (n) => String(n).replace(/\B(?=(\d{3})+(?!\d))/g, ',');
console.log(withCommas(1234567), withCommas(950));

const strong = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{8,}$/;
const samples = ['Dhaka2026', 'dhaka2026', 'Dk2026'];
console.log(samples.map((sample) => strong.test(sample)).join(' '));
```

Output

```
50, 450
1,234,567 950
true false false
```

- `(?<=৳)\d+` : শুধু টাকার চিহ্নের পরের সংখ্যা; ডলারেরটি বাদ, এবং চিহ্নটি ফলে আসেনি।
- **হাজারের কমা:** `\B` (শব্দের সীমানা নয় এমন জায়গা), যার পরে তিন-তিন অঙ্কের এক বা একাধিক দল এবং তারপর আর কোনো অঙ্ক নেই। দেখতে জটিল, তাই বাস্তবে `toLocaleString` (অধ্যায় ১১) সহজ ও পাঠযোগ্য; এটি শুধু Lookahead বোঝার জন্য।
- **Password-এর তিনটি Lookahead** একই অবস্থান থেকে তিনটি আলাদা শর্ত যাচাই করে। কাজ করে, কিন্তু ব্যর্থ হলে **কোন** শর্ত ভাঙল বলে না। অনুচ্ছেদ ১৭.৮-এ এর ভালো বিকল্প।

১৭.৫ Regex-এর Method এবং Flag-এর গভীরে

methods.js · JavaScript চলবে: Node.js

```
const log = 'ERROR 12:01 disk; WARN 12:05 cpu; ERROR 12:09 net';
const entry = /(?<level>ERROR|WARN) (?<time>\d{2}:\d{2}) (?<what>\w+)/g;

for (const match of log.matchAll(entry)) {
  console.log(match.index, match.groups.level, match.groups.what);
}

console.log(log.replace(entry, '$<time>=$<what>'));
console.log(log.replace(/\d{2}:\d{2}/g, (time) => time.replace(':', 'h')));
console.log('আম, কলা;লিচু  ,জাম'.split(/\s*[,;]\s*/));
```

Output

```
0 ERROR disk
18 WARN cpu
34 ERROR net
12:01=disk; 12:05=cpu; 12:09=net
ERROR 12h01 disk; WARN 12h05 cpu; ERROR 12h09 net
[ 'আম', 'কলা', 'লিচু', 'জাম' ]
```

- `matchAll` (`g` Flag বাধ্যতামূলক) প্রতিটি মিলের পূর্ণ তথ্য দেয়: অবস্থান (`index`), Group, Named Group। `match(/.../g)` শুধু মিলে যাওয়া লেখাগুলো দেয়।
- `replace`-এ `$<name>` Named Group-এর মান বসায় (`$1`, `$2` নম্বরের জন্য)।
- `replace`-এ **Function**: প্রতিটি মিলের জন্য Function ডাকা হয়, এবং তার ফেরত দেওয়া লেখা বসে। জটিল রূপান্তরে এটি সবচেয়ে পরিষ্কার।
- `split` **Regex নেয়:** কমা বা সেমিকোলন, চারপাশের যেকোনো পরিমাণ ফাঁকা জায়গাসহ, একবারে।

lastIndex-এর ফাঁদ

last-index.js · JavaScript চলবে: Browser Console ও Node.js

```
const digits = /\d+/g;

console.log(digits.test('abc 42'), digits.lastIndex);
console.log(digits.test('abc 42'), digits.lastIndex);
console.log(digits.test('abc 42'));

const safe = /\d+/;
console.log(safe.test('abc 42'), safe.test('abc 42'), safe.test('abc 42'));
```

Output

```
true 6
false 0
true
true true true
```

`g` বা `y` Flag-যুক্ত একটি Regex Object মনে রাখে শেষ মিল কোথায় শেষ হয়েছিল (`lastIndex`), এবং পরের `test` বা `exec` সেখান থেকে খোঁজা শুরু করে। একই লেখায় দ্বিতীয়বার `test` তাই `false` দিয়েছে, এবং শুরুতে ফিরে গেছে। এই Bug সাধারণত দেখা দেয় যখন একটি `g` Regex Module-এর একটি Constant-এ রেখে অনেক Input যাচাই করা হয়: পর্যায়ক্রমে সত্য-মিথ্যা। নিয়ম: **যাচাইয়ের (`test`) Regex-এ `g` দেবেন না।**

Flag	অর্থ
`m`	`^` ও `$` প্রতিটি লাইনের শুরু ও শেষ

Flag	অর্থ
`s`	`.` নতুন লাইনও মেলায়
`y` (Sticky)	ঠিক `lastIndex` থেকেই মিলতে হবে, পরে খোঁজা নয়; Tokenizer-এ কাজে লাগে
`d`	মিলের সঙ্গে প্রতিটি Group-এর শুরু-শেষ অবস্থান (`indices`)

ব্যবহারকারীর লেখা দিয়ে Regex: Escaping

খোঁজার Box-এ ব্যবহারকারী `c++` লিখলেন, আপনি লিখলেন `new RegExp(query)`। ফল: `+` একটি Quantifier, এবং `c++` অবৈধ বা অপ্রত্যাশিত Pattern।

escape.js · JavaScript — চলবে: Browser Console ও Node.js

```
const escapeRegExp = (text) => text.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');

const query = 'c++ (basic)';
const titles = ['c++ (basic) guide', 'c (basic)', 'cc (basic)'];

try {
  new RegExp(query);
} catch (error) {
  console.log(error.name);
}

const safe = new RegExp(escapeRegExp(query), 'i');
console.log(escapeRegExp(query));
console.log(titles.filter((title) => safe.test(title)).length);
```

Output

```
SyntaxError
c\+\+ \(basic\)
1
```

`escapeRegExp` প্রতিটি বিশেষ অক্ষরের আগে একটি `\` বসায় (`$&` মানে "পুরো মিল")। ফলে ব্যবহারকারীর লেখা আক্ষরিক অর্থে খোঁজা হয়। আরও নিরাপদ বিকল্প: আক্ষরিক খোঁজার জন্য Regex-ই ব্যবহার না করা— `includes` বা `indexOf` (অধ্যায় ৮-এর `findAll`)।

১৭.৬ Unicode এবং বাংলা

bengali.js · JavaScript চলবে: Node.js

```
const toAsciiDigits = (text) =>
  text.replace(/[০-৯]/g, (digit) => String(digit.charCodeAt(0) - 0x09e6));

console.log(toAsciiDigits('০১৭১২-৩৪৫৬৭৮'));

const banglaName = /^[\p{Script=Bengali}\s.]+$/u;
console.log(banglaName.test('মোঃ রফিক'), banglaName.test('Rafiq'),
    banglaName.test('রফিক১'));

const words = 'আমি JavaScript শিখি'.match(/[\p{L}\p{M}]+/gu);
console.log(words);
const smile = '\u{1F600}';
console.log('বই'.match(/\bবই\b/u), smile.match(/^.$/u)?.[0].length);
```

Output

```
01712-345678
true false true
[ 'আমি', 'JavaScript', 'শিখি' ]
null 2
```

- **বাংলা সংখ্যা:** `০` থেকে `৯` Unicode-এ পরপর (U+09E6 থেকে U+09EF), তাই অক্ষরের নম্বর থেকে `0x09e6` বাদ দিলে অঙ্কটি পাওয়া যায়। ব্যবহারকারী বাংলা Keyboard-এ নম্বর লিখলে যাচাইয়ের আগে এভাবে রূপান্তর করুন।
- **`\p{Script=Bengali}`** শুধু বাংলা লিপির অক্ষর, কার-চিহ্ন ও সংখ্যা মেলায়; `u` Flag বাধ্যতামূলক। লক্ষ করুন, `রফিক১` মিলেছে, কারণ বাংলা অঙ্কও বাংলা লিপির অংশ। নামে সংখ্যা না চাইলে আলাদা শর্ত লাগবে।
- **`\b` বাংলায় কাজ করে না,** এমনকি `u` Flag দিয়েও: `\b` এখনো `\w`-এর ইংরেজি সংজ্ঞায় চলে। বাংলা শব্দের সীমানার জন্য Lookaround লিখুন, যেমন `(?<![\p{L}\p{M}])বই(?![\p{L}\p{M}])`।
- **`u` Flag ছাড়া `.`** একটি Emoji-কে দুটি টুকরো ভাবে (অধ্যায় ৮-এর Code Unit)। `u` দিলে Code Point ধরে মেলে; তবু `length` এখনো ২, কারণ String-এর দৈর্ঘ্য Code Unit-এ।

১৭.৭ Normalize, Validate, Parse: একটি ক্রম

ব্যবহারকারীর Input-এর একই অর্থের বহু রূপ: `+880 1712-345678`, `8801712345678`, `০১৭১২ ৩৪৫৬৭৮`। প্রতিটি রূপের জন্য আলাদা Regex লেখা অসম্ভব। পদ্ধতিটি তিন ধাপের:

১. **Normalize:** একটি আদর্শ রূপে আনুন—Unicode NFC (অধ্যায় ৮), বাংলা অঙ্ক থেকে ইংরেজি, ফাঁকা জায়গা ও হাইফেন বাদ, দেশের Code বাদ।

২. **Validate:** আদর্শ রূপটি একটি সরল, কঠোর Pattern দিয়ে যাচাই করুন।

৩. **Parse:** বৈধ লেখা থেকে অর্থপূর্ণ মান তৈরি করুন (যেমন তারিখ থেকে `Date`), এবং অর্থগত শর্ত যাচাই করুন।

pipeline.js · JavaScript চলবে: Browser Console ও Node.js

```
const toAsciiDigits = (text) =>
  text.replace(/[০-৯]/g, (digit) => String(digit.charCodeAt(0) - 0x09e6));

function normalizePhone(raw) {
  const digits = toAsciiDigits(raw.normalize('NFC')).replace(/[\s\-().]/g, '');
  return digits.replace(/^(?:\+?880)/, '0');
}

const BD_MOBILE = /^01[3-9]\d{8}$/;

for (const raw of ['+880 1712-345678', '০১৭১২ ৩৪৫৬৭৮', '8801512345678',
    '01212345678']) {
  const phone = normalizePhone(raw);
  console.log(phone, BD_MOBILE.test(phone));
}
```

Output

```
01712345678 true
01712345678 true
01512345678 true
01212345678 false
```

যাচাইয়ের Pattern এখন অধ্যায় ৮-এর মতোই সরল; সব জটিলতা Normalization-এ, যেখানে প্রতিটি ধাপ আলাদাভাবে বোঝা ও পরীক্ষা করা যায়। সংরক্ষণও করুন আদর্শ রূপে, যাতে পরে খোঁজা ও তুলনা সহজ হয়।

Parse ধাপ কেন আলাদা, তারিখে দেখুন:

parse-date.js · JavaScript চলবে: Browser Console ও Node.js

```
const ISO_DATE = /^(?<y>\d{4})-(?<m>\d{2})-(?<d>\d{2})$/;

function parseDate(text) {
  const match = ISO_DATE.exec(text);
  if (!match) {
    return null;
  }
  const [y, m, d] = [match.groups.y, match.groups.m, match.groups.d].map(Number);
  const date = new Date(Date.UTC(y, m - 1, d));
  const real = date.getUTCMonth() === m - 1 && date.getUTCDate() === d;
  return real ? date.toISOString().slice(0, 10) : null;
}

const inputs = ['2026-10-15', '2026-02-30', '15/10/2026', '2024-02-29',
  '2026-02-29'];
console.log(inputs.map((text) => String(parseDate(text))).join(' '));
```

Output

```
2026-10-15 null null 2024-02-29 null
```

`2026-02-30` Regex পাস করে, কারণ গঠন ঠিক; কিন্তু ফেব্রুয়ারিতে ৩০ তারিখ নেই। `Date` এমন তারিখকে নীরবে মার্চে সরিয়ে দেয়, তাই তৈরি হওয়া তারিখের মাস ও দিন মূলটির সঙ্গে মিলিয়ে দেখা হয়েছে। Regex গঠন যাচাই করে, অর্থ নয়।

১৭.৮ Email, Phone এবং Password: Regex-এর সীমা

- **Email:** পূর্ণ মান অনুযায়ী Email-এর ব্যাকরণ এত জটিল যে তার Regex কয়েক হাজার অক্ষরের, আর তবু একটি ঠিকানা **আছে** কি না তা প্রমাণ করে না। ব্যবহারিক নিয়ম: একটি সরল গঠন-যাচাই (`@`-এর দুই পাশে কিছু, পরে একটি ডট), দৈর্ঘ্যের সীমা (২৫৪), এবং নিশ্চিত করতে একটি যাচাই-Email। বেশি কড়া Regex বৈধ ঠিকানাও বাতিল করে, যেমন `sara+bills@example.com`।
- **Phone:** আন্তর্জাতিক নম্বরে নিয়ম দেশে দেশে আলাদা; নিজের দেশের জন্য Normalize করে সরল Pattern, অন্য দেশের জন্য একটি মানসম্মত Library।
- **Name:** নামের কোনো সর্বজনীন নিয়ম নেই। অ্যাপোস্ট্রফি, হাইফেন, ডট, এক-শব্দের নাম—সব বৈধ। দৈর্ঘ্য ও নিয়ন্ত্রণ-অক্ষর (Control Character) বাদ দেওয়া যথেষ্ট।
- **Password:** একটি বড় Lookahead Regex ব্যবহারকারীকে শুধু "হয়নি" বলে। আলাদা আলাদা নিয়ম, প্রতিটির নিজের বার্তা, অনেক বেশি সহায়ক; আর আধুনিক নির্দেশিকা (যেমন NIST) জটিলতার নিয়মের চেয়ে **দৈর্ঘ্য** ও পরিচিত ফাঁস হওয়া Password-এর তালিকা যাচাইকে বেশি গুরুত্ব দেয়।

password.js · JavaScript চলবে: Node.js

```
const PASSWORD_RULES = [
  { test: (p) => p.length >= 12, message: 'কমপক্ষে 12 অক্ষর' },
  { test: (p) => /\p{Lu}/u.test(p), message: 'একটি বড় হাতের অক্ষর' },
  { test: (p) => /\d/.test(p), message: 'একটি সংখ্যা' },
  { test: (p) => !/(.)\1\1/.test(p), message: 'একই অক্ষর পরপর তিনবার নয়' },
];

const problems = (password) =>
  PASSWORD_RULES
    .filter((rule) => !rule.test(password))
    .map((rule) => rule.message);

console.log(problems('aaab'));
console.log(problems('Bangladesh2026!'));
```

Output

```
[
  'কমপক্ষে 12 অক্ষর',
  'একটি বড় হাতের অক্ষর',
  'একটি সংখ্যা',
  'একই অক্ষর পরপর তিনবার নয়'
]
[]
```

প্রতিটি নিয়ম একটি ছোট, নামযুক্ত পরীক্ষা; ব্যবহারকারী জানেন ঠিক কী বদলাতে হবে। Backreference `(.)\1\1` "একই অক্ষর তিনবার" ধরেছে।

১৭.৯ ReDoS: যখন একটি Regex Server আটকে দেয়

JavaScript-এর Regex Engine মেলাতে না পারলে পেছনে ফিরে অন্য পথ চেষ্টা করে (**Backtracking**)। কিছু Pattern-এ পথের সংখ্যা ইনপুটের দৈর্ঘ্যের সঙ্গে সূচকীয় হারে বাড়ে। এর সবচেয়ে পরিচিত রূপ Nested Quantifier: `(a+)+$`। প্রায়-মেলা একটি ইনপুট (`aaaa...ab`) দিলে Engine প্রতিটি সম্ভাব্য ভাগ চেষ্টা করে। JavaScript Single-threaded (অধ্যায় ১০), তাই এই সময়ে Server অন্য কোনো অনুরোধের উত্তর দেয় না। ইচ্ছাকৃতভাবে এমন ইনপুট পাঠানোকে বলে **ReDoS** (Regular Expression Denial of Service)।

ক্ষতিকর নয় এমন ছোট দৈর্ঘ্যে বৃদ্ধির হার মাপুন:

redos.js · JavaScript চলবে: Node.js

```
const evil = /^(a+)+$/;
const safe = /^a+$/;

function timeMs(regex, input) {
```

```
  const start = performance.now();
  regex.test(input);
  return performance.now() - start;
}

timeMs(evil, 'aaab');
timeMs(safe, 'aaab');

for (const n of [16, 18, 20, 22, 24]) {
  const ms = timeMs(evil, `${'a'.repeat(n)}b`);
  console.log(`/^(a+)+$/  ${n} অক্ষর: ${ms.toFixed(2)} ms`);
}
const safeMs = timeMs(safe, `${'a'.repeat(24)}b`);
console.log(`/^a+$/     24 অক্ষর: ${safeMs.toFixed(2)} ms`);
```

প্রথম দুটি ডাক Warm-up: প্রথমবার Regex Compile হওয়ার খরচ যাতে মাপে না ঢোকে (Benchmark-এর এই সতর্কতা অধ্যায় ১৯-এ বিস্তারিত)। সময়ের সংখ্যা যন্ত্র ও মুহূর্তভেদে বদলায়, তাই এই Listing-এর কোনো নির্দিষ্ট "প্রত্যাশিত" Output নেই, এবং বইয়ের স্বয়ংক্রিয় যাচাই একে শুধু চালায়, সংখ্যা মেলায় না। এই বইয়ের লেখার সময় একটি যন্ত্রে (Windows 10, Node.js 22.16.0) একবার চালানোর আসল Output:

Terminal

```
/^(a+)+$/  16 অক্ষর: 0.44 ms
/^(a+)+$/  18 অক্ষর: 1.73 ms
/^(a+)+$/  20 অক্ষর: 6.52 ms
/^(a+)+$/  22 অক্ষর: 28.40 ms
/^(a+)+$/  24 অক্ষর: 110.09 ms
/^a+$/     24 অক্ষর: 0.05 ms
```

সঙ্গে সঙ্গে দ্বিতীয়বার চালালে সংখ্যাগুলো সামান্য আলাদা ছিল (যেমন ১৬ অক্ষরে 0.58 ms, ২৪ অক্ষরে 111.46 ms), কিন্তু বৃদ্ধির হার একই: প্রতি দুটি অক্ষরে সময় প্রায় চারগুণ; এই হারে ৪০ অক্ষরে কয়েক ঘণ্টা। অথচ `^a+$` ঠিক একই ভাষা মেলায় এবং প্রায় সঙ্গে সঙ্গে শেষ হয়।

প্রতিরোধ:

- **Nested বা পরস্পর-আচ্ছাদিত Quantifier এড়ান:** `(a+)+`, `(a|a)*`, `(\w+\s?)*`।
- **Regex চালানোর আগে দৈর্ঘ্যের সীমা:** ১০০ অক্ষরের সীমায় বিপজ্জনক Pattern-ও ক্ষতি করতে পারে না। Project প্রতিটি ঘরে এটি প্রথমে যাচাই করে।
- **ব্যবহারকারীর লেখা দিয়ে কখনো Pattern নয়** (Escaping ছাড়া)।
- **Server-এ অজানা Pattern চালাতে হলে** Worker Thread-এ সময়সীমাসহ, বা Backtracking-মুক্ত Engine (যেমন RE2) দিয়ে।

১৭.১০ নিরাপদ Input প্রক্রিয়া: Regex কোথায় থামবে

- **Allow-list, Deny-list নয়:** "এগুলো বাদে সব চলবে" (যেমন `<script>` খুঁজে বাদ) সব সময় ভাঙা যায়: `<SCRIPT>`, `<scr<script>ipt>`, `<img onerror>`। "শুধু এগুলোই চলবে" (যেমন শুধু অক্ষর, সংখ্যা, ফাঁকা জায়গা) নিরাপদ।
- **Regex দিয়ে HTML পরিষ্কার করবেন না।** লেখা দেখাতে `textContent` (অধ্যায় ৮); যদি সত্যিই ব্যবহারকারীর HTML দেখাতে হয়, DOMPurify-এর মতো পরীক্ষিত Library, যা আসল HTML Parser ব্যবহার করে।
- **Validation ও Output Encoding আলাদা কাজ।** বৈধ Input-ও একটি প্রেক্ষাপটে বিপজ্জনক হতে পারে: `O'Brien` একটি বৈধ নাম, কিন্তু SQL-এ হাতে জোড়া দিলে Query ভাঙে। প্রতিটি প্রেক্ষাপটের নিজস্ব নিরাপদ উপায় আছে (অধ্যায় ২০)।
- **URL-এর জন্য `URL`** (অধ্যায় ১৫), তারিখের জন্য Parse ও অর্থগত যাচাই, সংখ্যার জন্য রূপান্তর ও সীমা—Regex শুধু গঠনের জন্য।
- **Client-এর যাচাই সুবিধা, নিরাপত্তা নয়।** একই Schema Server-এও চালান; এজন্য Project-এর Engine DOM-মুক্ত, Browser ও Node.js দুই জায়গাতেই চলে।

১৭.১১ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
Anchor ছাড়া যাচাই (`/\d{11}/`)	`abc01712345678xyz` গৃহীত	`^...$`
`^a|b$`	Alternation-এর অর্ধেক Anchor-হীন	`^(?:a|b)$`
যাচাইয়ের Regex-এ `g`	পর্যায়ক্রমে সত্য-মিথ্যা	`g` বাদ দিন
`.` দিয়ে "যেকোনো কিছু"	Greedy মিল অতিরিক্ত নেয়	Negated Class বা Lazy
Escaping ছাড়া `new RegExp(userText)`	`SyntaxError` বা ভুল মিল	`escapeRegExp`, বা `includes`
বাংলায় `\w` বা `\b`	বাংলা বাদ পড়ে	`\p{...}` এবং Lookaround
Normalize-এর আগে যাচাই	বাংলা অঙ্ক বা `+880` বাতিল	আগে Normalize
Regex পাস মানেই বৈধ	`2026-02-30` গৃহীত	Parse ও অর্থগত যাচাই
Nested Quantifier	লম্বা ইনপুটে আটকে যাওয়া	Pattern সরল করুন, দৈর্ঘ্যের সীমা
Regex দিয়ে HTML পরিষ্কার	XSS	`textContent` বা পরীক্ষিত Library

১৭.১২ Debugging নির্দেশিকা

- **ছোট টুকরো করে পরীক্ষা করুন:** পুরো Pattern কাজ না করলে Anchor সরিয়ে, তারপর এক এক Group যোগ করে দেখুন কোথায় মেলা থামে।
- **ইতিবাচক ও নেতিবাচক উদাহরণের তালিকা:** যা অবশ্যই মিলবে এবং যা অবশ্যই মিলবে না, দুটি তালিকাই রাখুন; এগুলোই পরে Unit Test।
- `matchAll` ও `d` **Flag** দিয়ে দেখুন ঠিক কোন অংশ কোন Group-এ গেছে।
- **Visual Tool:** regex101.com-এর মতো Website-এ Pattern-এর প্রতিটি অংশের ব্যাখ্যা ও ধাপে ধাপে মেলানো দেখা যায় (ECMAScript Flavor বেছে নিন)। সেখানে ব্যবহারকারীর সংবেদনশীল তথ্য পেস্ট করবেন না।
- **মাঝে মাঝে ভুল ফল?** `g` Flag ও `lastIndex` সন্দেহ করুন।
- **হঠাৎ ধীর?** Nested Quantifier খুঁজুন, এবং ইনপুটের দৈর্ঘ্য Log করুন।

১৭.১৩ পেশাদার অভ্যাস

- **নামযুক্ত Constant-এ Pattern,** পাশে একটি বাক্যে ব্যাখ্যা এবং কয়েকটি উদাহরণ।
- **Named Group** নম্বরের বদলে।
- **সব সময় Anchor** যাচাইয়ের Pattern-এ, Alternation Group-এর ভেতরে।
- **Normalize, তারপর Validate, তারপর Parse,** প্রতিটি ধাপ আলাদা Function।
- **দৈর্ঘ্যের সীমা আগে, Regex পরে।**
- **প্রতিটি নিয়মের নিজস্ব বার্তা,** একটি বড় Regex-এর বদলে।
- **একই Validation Code Client ও Server-এ।**
- **Regex শুধু গঠনের জন্য;** HTML, URL, তারিখ, সংখ্যার জন্য উপযুক্ত হাতিয়ার।

১৭.১৪ Project: FormShield

FormShield একটি ছোট, পুনর্ব্যবহারযোগ্য Validation Engine: প্রতিটি Form-এর জন্য একটি **Schema** লিখুন—কোন ঘর, কীভাবে Normalize, কোন নিয়মে যাচাই, কীভাবে Parse—এবং Engine সব ঘরের ফল ও সব ভুল একসঙ্গে দেয়। অধ্যায় ১১-এর OrderGuard একটি নির্দিষ্ট Form-এর জন্য হাতে লেখা যাচাই ছিল; FormShield যেকোনো Form-এর জন্য একটি সাধারণ যন্ত্র।

Project-এর চাহিদা

- প্রতিটি ঘরের Normalization: NFC, আগে-পরে ফাঁকা জায়গা বাদ, মাঝের বাড়তি ফাঁকা জায়গা একটিতে, বাংলা অঙ্ক থেকে ইংরেজি (যেখানে প্রযোজ্য)।
- **ReDoS প্রতিরোধ:** প্রতিটি ঘরের দৈর্ঘ্য Regex-এর **আগে** যাচাই; সব লেখা `MAX_INPUT` (২৫৬) অক্ষরে সীমিত।

- নিয়মগুলো ছোট, নামযুক্ত, প্রতিটির নিজস্ব বার্তা; একটি ঘরের প্রথম ভাঙা নিয়মের বার্তা জানানো।
- বাংলাদেশি মোবাইল, Email, ব্যবহারকারী-নাম, ISO তারিখ (অর্থগত যাচাইসহ), Password নীতি।
- Schema-র বাইরের ঘর বাদ (Mass Assignment প্রতিরোধ); `__proto__` ধরনের Key কখনো ফলে যায় না।
- DOM-মুক্ত, তাই Browser ও Node.js দুই জায়গাতেই একই নিয়ম।

File-এর কাঠামো

Folder-এর কাঠামো

```
form-shield/
├── src/
│   ├── normalize.mjs     লেখা ও সংখ্যা Normalization
│   ├── patterns.mjs      নামযুক্ত Regex Constant ও Escaping
│   ├── rules.mjs         নিয়মের Factory
│   └── engine.mjs        Schema চালানোর Engine
├── signup-schema.mjs     একটি Form-এর Schema (উদাহরণ)
├── demo.mjs
├── README.md
└── tests/
    ├── normalize.test.mjs
    ├── rules.test.mjs
    └── engine.test.mjs
```

src/normalize.mjs ও src/patterns.mjs

normalize.mjs · JavaScript — চলবে: Browser Console ও Node.js

```javascript
// বাংলা অঙ্ক (০-৯) থেকে ইংরেজি অঙ্ক।
export const toAsciiDigits = (text) =>
  text.replace(/[০-৯]/g, (digit) => String(digit.charCodeAt(0) - 0x09e6));

// NFC, আগে-পরে ফাঁকা জায়গা বাদ, মাঝের ফাঁকা জায়গা একটিতে।
export const cleanText = (value) =>
  String(value ?? '').normalize('NFC').replace(/\s+/g, ' ').trim();

export function normalizePhone(value) {
  const digits = toAsciiDigits(cleanText(value)).replace(/[\s\-().]/g, '');
  return digits.replace(/^\+?880/, '0');
}

export const normalizeEmail = (value) => cleanText(value).toLowerCase();
```

patterns.mjs · JavaScript চলবে: Browser Console ও Node.js

```
// প্রতিটি Pattern Anchor-সহ, কোনো g Flag নেই, কোনো Nested Quantifier নেই।
export const PATTERNS = Object.freeze({
  // 01, তারপর 3-9-এর একটি অঙ্ক, তারপর ঠিক ৪টি অঙ্ক: 01712345678
  BD_MOBILE: /^01[3-9]\d{8}$/,
  // @-এর দুই পাশে ফাঁকা জায়গা ও @ ছাড়া কিছু, পরে একটি ডট।
  EMAIL: /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
  // ইংরেজি ছোট হাতের অক্ষরে শুরু, তারপর 2-19টি অক্ষর, অঙ্ক বা _
  USERNAME: /^[a-z][a-z0-9_]{2,19}$/,
  // YYYY-MM-DD, অর্থগত যাচাই আলাদা (rules.mjs)
  ISO_DATE: /^(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})$/,
  // অক্ষর, কার-চিহ্ন, ফাঁকা জায়গা, ডট, অ্যাপোস্ট্রফি, হাইফেন
  PERSON_NAME: /^[\p{L}\p{M}][\p{L}\p{M} .'-]*$/u,
});

export const escapeRegExp = (text) => text.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
```

src/rules.mjs

rules.mjs · JavaScript চলবে: Browser Console ও Node.js

```
import { PATTERNS } from './patterns.mjs';

// প্রতিটি নিয়ম: (value) => বার্তা, অথবা ঠিক থাকলে null।
export const required = (message = 'ঘরটি পূরণ করুন।') => (value) =>
  (value === '' ? message : null);

export const length = (min, max) => (value) =>
  ([...value].length < min || [...value].length > max
    ? `দৈর্ঘ্য ${min} থেকে ${max} অক্ষরের মধ্যে হতে হবে।`
    : null);

export const matches = (regex, message) => (value) =>
  (regex.test(value) ? null : message);

export const notIn = (blocked, message) => (value) =>
  (blocked.includes(value.toLowerCase()) ? message : null);

export function realDate(value) {
  const match = PATTERNS.ISO_DATE.exec(value);
  if (!match) {
    return 'তারিখ YYYY-MM-DD আকারে লিখুন।';
  }
  const [y, m, d] = Object.values(match.groups).map(Number);
  const date = new Date(Date.UTC(y, m - 1, d));
  const real = date.getUTCMonth() === m - 1 && date.getUTCDate() === d;
  return real ? null : 'এমন কোনো তারিখ নেই।';
}
```

```
export const passwordPolicy = [
  length(12, 64),
  matches(/\p{Lu}/u, 'একটি বড় হাতের অক্ষর দিন।'),
  matches(/\d/, 'একটি সংখ্যা দিন।'),
  (value) => (/(.)\1\1/.test(value) ? 'একই অক্ষর পরপর তিনবার নয়।' : null),
];
```

প্রতিটি নিয়ম একটি **Factory** (অধ্যায় ১৪): প্রয়োজনীয় সেটিংস নিয়ে একটি ছোট Function ফেরত দেয়, যা একটি মান নিয়ে বার্তা বা `null` দেয়। `length` গোনে Code Point (`[...value]`), যাতে একটি Emoji দুটি অক্ষর না গোনা হয় (অধ্যায় ৮)। `realDate` হলো অনুচ্ছেদ ১৭.৭-এর Parse ধাপ: Named Group-এর মান `Object.values` দিয়ে ক্রমে বের করা হয়েছে।

src/engine.mjs

engine.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
import { cleanText } from './normalize.mjs';

export const MAX_INPUT = 256;

// Schema: { field: { normalize, rules: [...], optional } }
// ফল: { ok, values, errors }; values-এ শুধু Schema-র ঘর।
export function validate(schema, input) {
  const values = Object.create(null);
  const errors = Object.create(null);

  for (const [field, spec] of Object.entries(schema)) {
    const raw = Object.hasOwn(input ?? {}, field) ? input[field] : '';
    const text = String(raw ?? '');

    if (text.length > MAX_INPUT) {
      errors[field] = `লেখা অনেক বড় (সর্বোচ্চ ${MAX_INPUT} অক্ষর)।`;
      continue;
    }

    const value = (spec.normalize ?? cleanText)(text);
    if (spec.optional && value === '') {
      continue;
    }

    const message = spec.rules.map((rule) => rule(value)).find(Boolean);
    if (message) {
      errors[field] = message;
    } else {
      values[field] = value;
    }
  }
```

```
  const ok = Object.keys(errors).length === 0;
  return { ok, values: { ...values }, errors: { ...errors } };
}
```

- **দৈর্ঘ্য সবার আগে:** Normalization বা কোনো Regex চলার আগেই ২৫৬ অক্ষরের সীমা। একটি ১০ লক্ষ অক্ষরের ক্ষতিকর ইনপুট সঙ্গে সঙ্গে প্রত্যাখ্যাত (ReDoS প্রতিরোধ)।
- **শুধু Schema-র ঘর:** Loop চলে Schema-র ওপর, ইনপুটের ওপর নয়। ইনপুটে `isAdmin: true` বা `__proto__` থাকলেও ফলে যায় না—**Mass Assignment** প্রতিরোধ। `Object.hasOwn` উত্তরাধিকারসূত্রে পাওয়া Key পড়ে না (অধ্যায় ১১)।
- **`Object.create(null)`** একটি Prototype-হীন Object তৈরি করে, যাতে কোনো ঘরের নাম কোনো উত্তরাধিকারসূত্রে পাওয়া Property-র সঙ্গে সংঘর্ষে না যায়; শেষে সাধারণ Object-এ Copy করে ফেরত।
- **`find(Boolean)`:** নিয়মগুলো ক্রমে চলে, প্রথম বার্তাটি নেওয়া হয়। প্রথমে "পূরণ করুন", তারপর দৈর্ঘ্য, তারপর গঠন—সবচেয়ে মৌলিক সমস্যাটি আগে জানানো।
- **Optional ঘর** ফাঁকা হলে নিয়ম চলে না; পূরণ করলে চলে।

Schema এবং প্রদর্শনী

signup-schema.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
import { normalizeEmail, normalizePhone, cleanText } from './src/normalize.mjs';
import { PATTERNS } from './src/patterns.mjs';
import {
  length, matches, notIn, passwordPolicy, realDate, required,
} from './src/rules.mjs';

export const signupSchema = {
  name: {
    rules: [required(), length(2, 60), matches(PATTERNS.PERSON_NAME, 'নামে শুধু অক্ষর
        দিন।')],
  },
  username: {
    normalize: (value) => cleanText(value).toLowerCase(),
    rules: [
      required(),
      matches(PATTERNS.USERNAME, '3-20 অক্ষর: ইংরেজি ছোট হাতের অক্ষর, অঙ্ক বা _'),
      notIn(['admin', 'root', 'support'], 'এই নাম সংরক্ষিত।'),
    ],
  },
  email: { normalize: normalizeEmail, rules: [required(), matches(PATTERNS.EMAIL,
      'সঠিক Email লিখুন।')] },
  phone: {
    normalize: normalizePhone,
    rules: [required(), matches(PATTERNS.BD_MOBILE, 'সঠিক বাংলাদেশি মোবাইল নম্বর লিখুন।')],
```

```
  },
  birthDate: { optional: true, rules: [realDate] },
  password: {
    normalize: (value) => value,
    rules: [required(), ...passwordPolicy],
  },
};
```

demo.mjs · JavaScript চলবে: Node.js

```
import { validate } from './src/engine.mjs';
import { signupSchema } from './signup-schema.mjs';

const good = validate(signupSchema, {
  name: '  মোঃ  রফিক ',
  username: 'Rafiq_88',
  email: ' Rafiq@Example.COM ',
  phone: '+৮৮০ ১৭১২-৩৪৫৬৭৮',
  birthDate: '2000-02-29',
  password: 'Bogura-River-2026',
  isAdmin: true,
});
console.log(good.ok, good.values.name, good.values.phone, good.values.email);
console.log('isAdmin' in good.values, good.values.password === undefined);

const bad = validate(signupSchema, {
  name: 'R2D2',
  username: 'admin',
  email: 'rafiq@',
  phone: '01212345678',
  birthDate: '2001-02-29',
  password: 'aaa1111',
});
console.log(bad.ok, Object.keys(bad.errors).length);
for (const [field, message] of Object.entries(bad.errors)) {
  console.log(`${field}: ${message}`);
}
console.log(validate(signupSchema, { name: 'x'.repeat(1_000_000) }).errors.name);
```

Output

```
true মোঃ রফিক 01712345678 rafiq@example.com
false false
false 6
name: নামে শুধু অক্ষর দিন।
username: এই নাম সংরক্ষিত।
email: সঠিক Email লিখুন।
phone: সঠিক বাংলাদেশি মোবাইল নম্বর লিখুন।
birthDate: এমন কোনো তারিখ নেই।
password: দৈর্ঘ্য 12 থেকে 64 অক্ষরের মধ্যে হতে হবে।
লেখা অনেক বড় (সর্বোচ্চ 256 অক্ষর)।
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **বৈধ ইনপুট পরিষ্কার হয়ে আসে:** নামের বাড়তি ফাঁকা জায়গা, Email-এর বড় হাতের অক্ষর, `+৮৮০` ও বাংলা অঙ্কসহ মোবাইল নম্বর—সব আদর্শ রূপে। `+৮৮০`-এর বাংলা অঙ্ক আগে ইংরেজি হয়েছে, তারপর `880` বাদ পড়েছে; Normalization-এর ক্রম গুরুত্বপূর্ণ।
- `isAdmin` **ফলে নেই:** Schema-তে নেই এমন ঘর কখনো ফলের অংশ হয় না।
- **Password ফলে আছে** (`undefined` নয়), কারণ এই Engine-এর কাজ যাচাই; Password-এর Hash তৈরি ও নিরাপদ সংরক্ষণ অধ্যায় ২০-এর বিষয়। তার `normalize` লেখাটি অপরিবর্তিত রাখে, কারণ Password-এর ফাঁকা জায়গাও অর্থবহ।
- **২০০০ সাল অধিবর্ষ,** তাই ২৯ ফেব্রুয়ারি বৈধ; ২০০১-এর নয়।
- **ছয়টি ঘরের প্রতিটির নিজস্ব বার্তা,** একসঙ্গে। `R2D2` নামে সংখ্যা থাকায় বাতিল; `admin` গঠনে ঠিক, কিন্তু সংরক্ষিত তালিকায়।
- **দশ লক্ষ অক্ষরের নাম** Normalization বা Regex পর্যন্ত পৌঁছায়নি।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node demo.mjs
node --test
```

Companion Source Code-এ ২৪টি Test তিনটি File-এ: Normalization (বাংলা অঙ্ক, `+880`, NFC), নিয়ম (Boundary, অধিবর্ষ, Lookahead-মুক্ত Password নীতি, `g` Flag না থাকা), এবং Engine (Mass Assignment, `__proto__`, Optional ঘর, ReDoS-এর দৈর্ঘ্য-সীমা এবং ১০ লক্ষ অক্ষরের ইনপুটে তাৎক্ষণিক প্রত্যাখ্যান)।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
বাংলা অঙ্কের নম্বর বাতিল	Normalization-এ `toAsciiDigits` নেই	`normalize` ঘরে দিন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
একই ইনপুট কখনো বৈধ, কখনো নয়	Pattern-এ `g` Flag	`g` বাদ দিন
`2026-02-30` গৃহীত	শুধু Regex, Parse নেই	`realDate`
অজানা ঘর ফলে আসে	ইনপুটের ওপর Loop	Schema-র ওপর Loop
বড় ইনপুটে ধীর	দৈর্ঘ্যের সীমা পরে	`MAX_INPUT` আগে

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। Email ও নামের যাচাই ইচ্ছা করে উদার; ফাঁস হওয়া Password-এর তালিকা যাচাই নেই; বার্তাগুলো একটি ভাষায়। Browser-এ যাচাই শুধু ব্যবহারকারীর সুবিধা; Server-এ একই Schema আবার চালাতে হবে।

১৭.১৫ অনুশীলনী

প্রাথমিক স্তর

১. একটি লেখা থেকে সব `#hashtag` (ইংরেজি অক্ষর, অঙ্ক, `_`) বের করুন।

২. `^(?:jpg|png|gif)$` এবং `^jpg|png|gif$`-এর পার্থক্য `'xpng'` দিয়ে দেখান।

৩. Named Group দিয়ে `'14:35'` থেকে ঘণ্টা ও মিনিট আলাদা করুন।

৪. একটি Constant-এ রাখা `/\d+/g` দিয়ে একই ইনপুট তিনবার যাচাই করলে কী হয়? ঠিক করুন।

৫. বাংলা অঙ্কের একটি দাম (`'৳১,২৫০'`) থেকে সংখ্যা (`1250`) বের করুন।

মধ্যম স্তর

৬. `replace`-এ Function দিয়ে একটি লেখার প্রতিটি শব্দের প্রথম অক্ষর বড় হাতের করুন।

৭. `escapeRegExp` ব্যবহার করে ব্যবহারকারীর খোঁজার শব্দ (`'1+1=2'`) লেখায় চিহ্নিত করুন (`[1+1=2]`)।

৮. Lookbehind দিয়ে শুধু `#` চিহ্নের পরের Hex রঙের কোড (`#1d4f9c`) বের করুন, `#` ছাড়া।

৯. FormShield-এর জন্য একটি `postcode` নিয়ম লিখুন: বাংলাদেশি পোস্টকোড ৪ অঙ্ক, বাংলা অঙ্কও চলবে।

চ্যালেঞ্জ স্তর

১০. Sticky Flag (`y`) দিয়ে একটি ছোট Tokenizer লিখুন, যা `'12 + 3*4'` থেকে সংখ্যা ও চিহ্নের তালিকা দেয়, ফাঁকা জায়গা বাদ।

১১. একটি Log থেকে `matchAll` দিয়ে প্রতিটি স্তরের (ERROR, WARN) সংখ্যা গুনুন।

১২. FormShield-এ একটি `confirmPassword` ঘর যোগ করুন, যা `password`-এর সঙ্গে মিলতে হবে। Engine-এর নকশা কীভাবে বাড়াতে হবে, দেখান।

১৭.১৬ অনুশীলনীর সমাধান

সমাধান ১

hashtags.js · JavaScript — চলবে: Node.js

```js
const post = 'আজ #JavaScript আর #node_js শিখলাম! #2026 #';

console.log(post.match(/#\w+/g));
```

Output

```
[ '#JavaScript', '#node_js', '#2026' ]
```

একা `#`-এর পরে কোনো `\w` নেই, তাই মেলেনি (`+` মানে কমপক্ষে একটি)।

সমাধান ২

anchor-group.js · JavaScript — চলবে: Browser Console ও Node.js

```js
console.log(/^(?:jpg|png|gif)$/.test('xpng'));
console.log(/^jpg|png|gif$/.test('xpng'));
```

Output

```
false
true
```

দ্বিতীয় Pattern আসলে "`^jpg` অথবা `png` অথবা `gif$`", তাই `xpng`-এর ভেতরের `png` মিলেছে।

সমাধান ৩

time-groups.js · JavaScript — চলবে: Browser Console ও Node.js

```js
const { hour, minute } = /^(?<hour>\d{2}):(?<minute>\d{2})$/.exec('14:35').groups;

console.log(Number(hour) * 60 + Number(minute));
```

Output

```
875
```

সমাধান ৪ — `g`-যুক্ত Regex `lastIndex` মনে রাখে, তাই ফল হয় `true`, `false`, `true` (অনুচ্ছেদ ১৭.৫)। যাচাইয়ে `g` বাদ দিন: `/\d+/`। প্রতিটি ডাকে নতুন Regex তৈরি করাও কাজ করে, কিন্তু `g` বাদ দেওয়াই সঠিক সমাধান।

সমাধান ৫

bangla-price.js · JavaScript — চলবে: Browser Console ও Node.js

```
const toAsciiDigits = (text) =>
  text.replace(/[০-৯]/g, (digit) => String(digit.charCodeAt(0) - 0x09e6));

const price = Number(toAsciiDigits('৳১,২৫০').replace(/[^\d]/g, ''));
console.log(price + 1);
```

Output

```
1251
```

সমাধান ৬

title-case.js · JavaScript — চলবে: Browser Console ও Node.js

```
const titleCase = (text) =>
  text.toLowerCase().replace(/\b[a-z]/g, (letter) => letter.toUpperCase());

console.log(titleCase('modern JAVASCRIPT for bangla readers'));
```

Output

```
Modern Javascript For Bangla Readers
```

সমাধান ৭

highlight-escaped.js · JavaScript — চলবে: Browser Console ও Node.js

```
const escapeRegExp = (text) => text.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');

const query = '1+1=2';
const pattern = new RegExp(escapeRegExp(query), 'g');

console.log('প্রমাণ: 1+1=2, আর 11=2 নয়'.replace(pattern, (match) => `[${match}]`));
```

Output

```
প্রমাণ: [1+1=2], আর 11=2 নয়
```

Escaping ছাড়া `1+1` মানে "এক বা একাধিক 1, তারপর 1", এবং ভুল জায়গায় মিলত।

সমাধান ৮

hex-colors.js · JavaScript চলবে: Node.js

```
const css = 'color: #1d4f9c; background: #FFF; border: 1px solid #12305f80;';

console.log(css.match(/(?<=#)(?:[0-9a-f]{6}|[0-9a-f]{3})(?![0-9a-f])/gi));
```

Output

```
[ '1d4f9c', 'FFF' ]
```

শেষের Negative Lookahead নিশ্চিত করে যে ৮ অঙ্কের `#12305f80` থেকে প্রথম ৬টি ভুল করে নেওয়া হয়নি।

সমাধান ৯

postcode.mjs · JavaScript চলবে: Node.js

```
import { validate } from '../../project/form-shield/src/engine.mjs';
import {
  toAsciiDigits, cleanText,
} from '../../project/form-shield/src/normalize.mjs';
import { matches, required } from '../../project/form-shield/src/rules.mjs';

const schema = {
  postcode: {
    normalize: (value) => toAsciiDigits(cleanText(value)),
    rules: [required(), matches(/^\d{4}$/, 'পোস্টকোড 4 অঙ্কের।')],
  },
};

for (const postcode of ['৫৮০০', '5800', '580', 'abcd']) {
  const result = validate(schema, { postcode });
  const shown = result.ok ? result.values.postcode : result.errors.postcode;
  console.log(postcode, shown);
}
```

Output

```
৫৮০০ 5800
5800 5800
580 পোস্টকোড 4 অঙ্কের।
abcd পোস্টকোড 4 অঙ্কের।
```

Engine-এর কোনো পরিবর্তন লাগেনি; নতুন ঘর মানে শুধু Schema-য় একটি নতুন প্রবেশ।

সমাধান ১০

tokenizer.js · JavaScript চলবে: Node.js

```
function tokenize(text) {
  const token = /\s*(?:(?<number>\d+)|(?<op>[+\-*/]))/y;
  const source = text.trimEnd();
  const tokens = [];
  let position = 0;

  while (position < source.length) {
    token.lastIndex = position;
    const match = token.exec(source);
    if (match === null) {
      throw new SyntaxError(`অচেনা অক্ষর, অবস্থান ${position}`);
    }
    tokens.push(match.groups.number ?? match.groups.op);
    position = token.lastIndex;
  }
  return tokens;
}

console.log(tokenize('12 + 3*4'));
try {
  tokenize('12 + x');
} catch (error) {
  console.log(error.message);
}
```

Output

```
[ '12', '+', '3', '*', '4' ]
অচেনা অক্ষর, অবস্থান 4
```

Sticky Flag-এ প্রতিটি মিল ঠিক `lastIndex` থেকে শুরু হতে হয়; অচেনা অক্ষরে এসে মেলা থামে, এবং অবস্থান থেকে বোঝা যায় কোথায় সমস্যা। মেলা ব্যর্থ হলে `exec` `lastIndex`-কে ০-তে ফিরিয়ে দেয়, তাই অবস্থান আলাদা Variable-এ (`position`) রাখা হয়েছে। `g` হলে Regex অচেনা অক্ষর লাফিয়ে পরের সংখ্যা খুঁজত, এবং ভুলটি নীরবে চাপা পড়ত।

সমাধান ১১

log-levels.js · JavaScript চলবে: Node.js

```
const log = `ERROR db timeout
WARN slow query
ERROR db timeout
INFO started
WARN cache miss`;

const counts = {};
for (const { groups } of log.matchAll(/^(?<level>ERROR|WARN)\b/gm)) {
  counts[groups.level] = (counts[groups.level] ?? 0) + 1;
}
console.log(counts);
```

Output

```
{ ERROR: 2, WARN: 2 }
```

`m` Flag-এ `^` প্রতিটি লাইনের শুরু; তাই একটি লাইনের মাঝের "ERROR" শব্দ গোনা হতো না।

সমাধান ১২

confirm.mjs · JavaScript চলবে: Node.js

```
import { validate } from '../../project/form-shield/src/engine.mjs';
import { required } from '../../project/form-shield/src/rules.mjs';

// ঘরের নিয়ম একটি মান দেখে; দুটি ঘরের তুলনা Engine-এর পরে, Form-স্তরের নিয়মে।
function validateWithChecks(schema, input, checks) {
  const result = validate(schema, input);
  const errors = { ...result.errors };
  for (const check of checks) {
    const problem = check(result.values);
    if (problem && !errors[problem.field]) {
      errors[problem.field] = problem.message;
    }
  }
  return { ok: Object.keys(errors).length === 0, values: result.values, errors };
}

const schema = {
  password: { normalize: (v) => v, rules: [required()] },
  confirmPassword: { normalize: (v) => v, rules: [required()] },
};
const samePassword = (values) =>
  (values.password !== values.confirmPassword
```

```
    ? { field: 'confirmPassword', message: 'দুটি Password মেলেনি।' }
    : null);

const input = {
  password: 'Bogura-River-2026',
  confirmPassword: 'Bogura-river-2026',
};
console.log(validateWithChecks(schema, input, [samePassword]).errors);
```

Output

```
{ confirmPassword: 'দুটি Password মেলেনি।' }
```

নকশার সিদ্ধান্ত: ঘরের নিয়ম শুধু নিজের মান দেখে, তাই সেগুলো সরল ও পুনর্ব্যবহারযোগ্য থাকে। একাধিক ঘরের সম্পর্ক (মেলা, তারিখের ক্রম) আলাদা "Form-স্তরের" নিয়মে, যা সব ঘর পরিষ্কার হওয়ার পর চলে।

১৭.১৭ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `/<.+?>/`-এর `?` কী বোঝায়?

ক) আগের অক্ষরটি ঐচ্ছিক

খ) Quantifier Lazy: যতটা সম্ভব কম নেয়

গ) Lookahead

ঘ) Named Group

প্রশ্ন ২. `/^cat|dog$/.test('catalog')`-এর ফল কেন `true`?

ক) `|`-এর অগ্রাধিকার সবচেয়ে কম; `^cat` একাই মেলে

খ) `i` Flag নেই

গ) `catalog`-এ `dog` আছে

ঘ) এটি `false` দেয়

প্রশ্ন ৩. `(?<year>\d{4})`-এর মান কোথায় পাওয়া যায়?

ক) `match.year`

খ) `match[0]`

গ) `match.groups.year`

ঘ) `match.index`

প্রশ্ন ৪. `(?<=৳)\d+` কোনটি মেলায়?

ক) `৳`-সহ পুরো দাম

খ) `৳`-এর আগের সংখ্যা

গ) যেকোনো সংখ্যা

ঘ) `৳`-এর পরের সংখ্যা, `৳` ছাড়া

প্রশ্ন ৫. একটি Constant-এ রাখা `/x/g` দিয়ে একই লেখা বারবার `test` করলে কী হয়?

ক) সব সময় `true`

খ) `lastIndex`-এর কারণে ফল পর্যায়ক্রমে বদলায়

গ) `SyntaxError`

ঘ) সব সময় `false`

প্রশ্ন ৬. ব্যবহারকারীর লেখা `c++` দিয়ে নিরাপদে খুঁজতে কী করবেন?

ক) `new RegExp('c++')`

খ) `i` Flag যোগ

গ) বিশেষ অক্ষর Escape করা, বা `includes` ব্যবহার

ঘ) `g` Flag যোগ

প্রশ্ন ৭. বাংলা লেখায় `\b` কেন নির্ভরযোগ্য নয়?

ক) `\b` শুধু ইংরেজি `\w`-এর সংজ্ঞায় শব্দের সীমানা চেনে

খ) বাংলায় ফাঁকা জায়গা নেই

গ) `\b` শুধু সংখ্যা চেনে

ঘ) `u` Flag `\b` নিষিদ্ধ করে

প্রশ্ন ৮. `2026-02-30` তারিখ-Regex পাস করে। সঠিক পদক্ষেপ?

ক) আরও জটিল Regex

খ) গ্রহণ করা

গ) `g` Flag

ঘ) Parse করে মাস ও দিন মিলিয়ে অর্থগত যাচাই

প্রশ্ন ৯. কোন Pattern ReDoS-এর ঝুঁকিপূর্ণ?

ক) `^(a+)+$`

খ) `^a+$`

গ) `^\d{4}$`

ঘ) `^[a-z]{3,20}$`

প্রশ্ন ১০. ReDoS প্রতিরোধে Regex চালানোর আগে কোনটি সবচেয়ে কার্যকর?

ক) `i` Flag

খ) ইনপুটের দৈর্ঘ্যের সীমা

গ) `toUpperCase`

ঘ) `JSON.stringify`

১৭.১৮ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–ক · ৩–গ · ৪–ঘ · ৫–খ · ৬–গ · ৭–ক · ৮–ঘ · ৯–ক · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	Quantifier-এর পরে `?` তাকে Lazy করে।
২	ক	`^cat` অথবা `dog$`; পুরোটাকে Group-এ রাখতে হয়।
৩	গ	Named Group-এর মান `groups` Object-এ।
৪	ঘ	Lookbehind শর্ত যাচাই করে, অক্ষর নেয় না।
৫	খ	`g` Regex শেষ অবস্থান মনে রাখে।
৬	গ	`+` একটি Quantifier; আক্ষরিক খোঁজায় Escape বা `includes`।
৭	ক	`u` Flag দিলেও `\b` বাংলা অক্ষরকে শব্দ-অক্ষর ধরে না।
৮	ঘ	Regex গঠন দেখে, অর্থ নয়।
৯	ক	Nested Quantifier-এ পথের সংখ্যা সূচকীয় হারে বাড়ে।
১০	খ	সীমিত দৈর্ঘ্যে সবচেয়ে খারাপ Pattern-ও দ্রুত শেষ হয়।

১৭.১৯ অধ্যায়ের সারসংক্ষেপ

- Character Class, Negated Class, `{m,n}`, `\b`; Quantifier Greedy, `?` দিলে Lazy।
- Group ধরে রাখে, Named Group নাম দেয়, `(?:)` শুধু দলবদ্ধ করে, `\1` আগের মিলের পুনরাবৃত্তি; Alternation-এর অগ্রাধিকার সবচেয়ে কম।
- Lookahead ও Lookbehind অবস্থানের শর্ত, অক্ষর গ্রাস করে না।
- `matchAll` পূর্ণ মিল-তথ্য, `replace`-এ `$<name>` ও Function, `split`-এ Regex, Sticky Tokenizer।
- `g` / `y` Regex `lastIndex` মনে রাখে; যাচাইয়ে `g` নয়।
- ব্যবহারকারীর লেখা দিয়ে Regex মানে Escaping; আক্ষরিক খোঁজায় `includes`।

- বাংলায় `\p{Script=Bengali}`, বাংলা অঙ্কের রূপান্তর; `\w` ও `\b` বাংলা চেনে না।
- Normalize, Validate, Parse—তিন আলাদা ধাপ; Regex শুধু গঠন যাচাই করে।
- Email, নাম, Password-এ Regex-এর বাড়াবাড়ি নয়; Password-এ আলাদা নিয়ম ও বার্তা।
- ReDoS: Nested Quantifier এড়ান, দৈর্ঘ্যের সীমা আগে; HTML পরিষ্কারে Regex নয়।

পরের অধ্যায়ে Functional Programming-এর উন্নত প্রয়োগ: Immutable তথ্য, জটিল তথ্যের রূপান্তর, Grouping ও Aggregation।

১৭.২০ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Character Class	বন্ধনীর ভেতরের যেকোনো একটি অক্ষর।
Quantifier	কতবার: `*`, `+`, `?`, `{m,n}`।
Greedy / Lazy	যতটা সম্ভব বেশি / কম নেওয়া।
Anchor	অবস্থান: `^`, `$`, `\b`।
Capturing Group	মিলে যাওয়া অংশ ধরে রাখা বন্ধনী।
Named Group	নামযুক্ত Capturing Group।
Backreference	আগের Group-এর মিলের পুনরাবৃত্তি।
Lookaround	পাশে কী আছে তার শর্ত, অক্ষর না নিয়ে।
Sticky Flag	ঠিক `lastIndex` থেকেই মেলানো।
Escaping	বিশেষ অক্ষরকে আক্ষরিক অর্থ দেওয়া।
ReDoS	Backtracking-এর বিস্ফোরণে Program আটকে যাওয়া।
Mass Assignment	অনাকাঙ্ক্ষিত ঘর ইনপুট থেকে ফলে ঢুকে পড়া।

১৭.২১ আরও পড়ুন

- **MDN — Regular expressions:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions` — সম্পূর্ণ নির্দেশিকা।
- **MDN — Lookahead assertion:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Lookahead_assertion` — Lookaround।

- **MDN — Unicode character class escape:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Unicode_character_class_escape` — `\p{...}`।
- **OWASP — Regular expression Denial of Service:** `https://owasp.org/www-community/attacks/Regular_expression_Denial_of_Service_-_ReDoS` — ReDoS-এর ব্যাখ্যা।
- **OWASP — Input Validation Cheat Sheet:** `https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html` — Allow-list ও যাচাইয়ের নীতি।